Logic, Counting, and Probabilities

Pedro Zuidberg Dos Martires

wait!

the basics first

The "***product***" rule:

$$p(\mathbf{X}, \mathbf{Y}, \mathbf{Z}) = p(\mathbf{X})p(\mathbf{Y}, \mathbf{Z} \mid \mathbf{X}) = p(\mathbf{X})p(\mathbf{Y} \mid \mathbf{X})p(\mathbf{Z} \mid \mathbf{X}, \mathbf{Y})$$

The "***sum***" rule:

$$p(\mathbf{X}, \mathbf{Y}) = \int_{\mathsf{dom}(\mathbf{z})} p(\mathbf{X}, \mathbf{Y}, \mathbf{z})\, d\mathbf{Z}$$

Propositional Logic

Natural language

An image contains either an elephant or a giraffe, but not both.

Propositional variables

$$e := \text{“The image contains an elephant”}$$

$$g := \text{“The image contains a giraffe”}$$

Logical representation

$$(e \vee g) \ \wedge \ \neg(e \wedge g)$$

Modelling the Real World

All models are wrong some are useful.

– George E. P. Box

Modelling the Real World

All models are wrong some are useful.

– George E. P. Box

No language is perfect some are useful.

Syntax of Propositional Logic

- literals: a, b, $hello$
- atoms: atoms and their negation $\neg a$, $\neg b$, $\neg hello$.
- connectives: $\wedge$ (AND), $\vee$ (OR)
- sentence/formula/logic statement: $(a \vee b) \wedge \neg(\neg b \wedge c)$

Normal forms:
conjunctive normal form (CNF): $\phi_c = (a \vee b) \wedge (b \vee \neg c)$
disjunctive normal form (DNF): $\phi_d = (a \wedge b) \vee (b \wedge \neg c)$

Semantics of Propositional Logic

What do the formulas mean?

a	$\neg a$
$\top$	$\bot$
$\bot$	$\top$

a	b	$a \wedge b$
$\top$	$\top$	$\top$
$\top$	$\bot$	$\bot$
$\bot$	$\top$	$\bot$
$\bot$	$\top$	$\bot$

a	b	$a \vee b$
$\top$	$\top$	$\top$
$\top$	$\bot$	$\top$
$\bot$	$\top$	$\top$
$\bot$	$\top$	$\bot$

Semantics of Propositional Logic

What do the formulas mean?

a	$\neg a$
$\top$	$\bot$
$\bot$	$\top$

a	b	$a \wedge b$
$\top$	$\top$	$\top$
$\top$	$\bot$	$\bot$
$\bot$	$\top$	$\bot$
$\bot$	$\top$	$\bot$

a	b	$a \vee b$
$\top$	$\top$	$\top$
$\top$	$\bot$	$\top$
$\bot$	$\top$	$\top$
$\bot$	$\top$	$\bot$

$$\phi_c = (a \vee b) \wedge (b \vee \neg c)$$

a	b	c	ϕ_c
$\bot$	$\bot$	$\bot$	$\bot$
$\bot$	$\bot$	$\top$	$\bot$
$\bot$	$\top$	$\bot$	$\top$
$\bot$	$\top$	$\top$	$\top$
$\top$	$\bot$	$\bot$	$\top$
$\top$	$\bot$	$\top$	$\bot$
$\top$	$\top$	$\bot$	$\top$
$\top$	$\top$	$\top$	$\top$

Satisfiability aka SAT

Is there a satisfying solution?

$$\phi = (a \vee b) \wedge (b \vee \neg c)$$

Satisfiability aka SAT

Is there a satisfying solution?

$$\phi = (a \vee b) \wedge (b \vee \neg c)$$

$\{a = \bot, b = \bot, c = \top\}$ NO

Satisfiability aka SAT

Is there a satisfying solution?

$$\phi = (a \vee b) \wedge (b \vee \neg c)$$

$\{a = \bot, b = \bot, c = \top\}$ NO

$\{a = \top, b = \bot, c = \bot\}$ YES

Satisfiability aka SAT

Is there a satisfying solution?

$$\phi = (a \vee b) \wedge (b \vee \neg c)$$

$\{a = \bot, b = \bot, c = \top\}$ NO

$\{a = \top, b = \bot, c = \bot\}$ YES

We found a solution: ϕ is satisfiable.
A solution is called a ”model” or ”possible world”.

Model Counting aka #SAT

$$\phi = (a \vee b) \wedge (b \vee \neg c)$$

Question:
How many satisfying worlds are there?

SAT:

$\exists x \ : \ x \models \phi$ there exists a variable assignment such that ϕ is satisfied

#SAT:

$\#SAT(\phi) = |\{x \mid x \models \phi\}|$ the cardinality of the set of satisfying assignments

Idea: Count the worlds satisfying the formula.

Model Counting aka #SAT

How many satisfying solutions are there?

$$\phi = (a \vee b) \wedge (b \vee \neg c)$$

Model Counting aka #SAT

$$\phi = (a \vee b) \wedge (b \vee \neg c)$$

a	b	c	MC(ϕ)
$\bot$	$\bot$	$\bot$	
$\bot$	$\bot$	$\top$	
$\bot$	$\top$	$\bot$	1
$\bot$	$\top$	$\top$	1
$\top$	$\bot$	$\bot$	1
$\top$	$\bot$	$\top$	
$\top$	$\top$	$\bot$	1
$\top$	$\top$	$\top$	1
			5

A Useful Ratio

Let

$$\phi = (a \vee b) \wedge (b \vee \neg c).$$

There are

$$\#SAT(\phi) = 5$$

satisfying worlds.

Three of them satisfy a. Therefore

$$\frac{\#SAT(a \wedge \phi)}{\#SAT(\phi)} = \frac{3}{5}.$$

Interpretation:

> *Among the worlds satisfying ϕ, what fraction also satisfy a?*

Conditional Probability

The ratio

$$\frac{\#SAT(\psi \wedge \phi)}{\#SAT(\phi)}$$

appears so often that we give it a name.

Assuming all worlds are equally likely,

$$p(\psi \mid \phi) = \frac{\#SAT(\psi \wedge \phi)}{\#SAT(\phi)}.$$

- Conditional probability is simply a normalized count of worlds.
- What is the probability of ψ being true given that ϕ is true?
- How much bigger is one set than the other set?

The Problem with Counting

Counting assumes that every world is equally likely.

$$p(x_1) = p(x_2) = \cdots = p(x_n)$$

But in reality:

$$p(\text{rain}) \neq p(\text{no rain})$$

Some worlds should contribute more than others.

$$\text{Counting} \longrightarrow \text{Weighted Counting}$$

Weighted Model Counting

What if the Boolean variables are **true** with a **probability**?

Weighted Model Counting

What if the Boolean variables are **true** with a **probability**?

What is the **probability** of the formula ϕ being **true**?
$p(a)=0.3$, $p(b)=0.1$, $p(c)=0.6$

Weighted Model Counting

What if the Boolean variables are **true** with a **probability**?

What is the **probability** of the formula ϕ being **true**?
$p(a)=0.3$, $p(b)=0.1$, $p(c)=0.6$

a	b	c	WMC(ϕ)
$\bot$	$\bot$	$\bot$	
$\bot$	$\bot$	$\top$	
$\bot$	$\top$	$\bot$	$(1-0.3)\times0.1\times(1-0.6)=0.028$
$\bot$	$\top$	$\top$	$(1-0.3)\times0.1\times0.6=0.042$
$\top$	$\bot$	$\bot$	$0.3\times(1-0.1)\times(1-0.6)=0.108$
$\top$	$\bot$	$\top$	
$\top$	$\top$	$\bot$	$0.3\times0.1\times(1-0.6)=0.012$
$\top$	$\top$	$\top$	$0.3\times0.1\times0.6=0.018$
			0.208

A Useful Ratio, Revisited

Recall:

$$p(\psi \mid \phi) = \frac{\#SAT(\psi \wedge \phi)}{\#SAT(\phi)}$$

when all worlds are equally likely.

If worlds have different weights, we replace counting by weighted counting:

$$p(\psi \mid \phi) = \frac{\mathrm{WMC}(\psi \wedge \phi)}{\mathrm{WMC}(\phi)}$$

Conditional probability is a normalized weighted count.

Probability Axioms

Let Ω denote the set of all possible worlds.

A probability measure p satisfies:

1. **Non-negativity**

$$p(A) \geq 0 \qquad \forall A \subseteq \Omega$$

2. **Normalization**

$$p(\Omega) = 1$$

3. **Finite Additivity**
   For pairwise disjoint events $A_1, \ldots, A_n$,

$$p\left(\bigcup_{i=1}^{n} A_i\right) = \sum_{i=1}^{n} p(A_i).$$

Weighted Model Counting as Probability

Let $w(x) \geq 0$ be the weight of a possible world x.

Define $p(\phi) = \sum_{x \models \phi} w(x)$.

If $\sum_{x \in \Omega} w(x) = 1$, then:

1. $p(\phi) \geq 0$
2. $p(\Omega) = 1$
3. $p(\phi \vee \psi) = p(\phi) + p(\psi)$ for disjoint events

Therefore WMC is a probability measure.

Probabilities as Weighted Counting

Assign each variable x_i a weight function

$$w(x_i)$$

and define the weight of a world $\mathbf{x} = (x_1, \ldots, x_n)$ as

$$w(\mathbf{x}) = \prod_{x_i \in \mathbf{x}} w(x_i).$$

The probability of a logical formula ϕ is

$$p(\phi) = \sum_{x \models \phi} w(x) = \sum_{x \models \phi} \prod_{x_i \in \mathbf{x}} w(x_i).$$

Conditional probabilities become

$$p(\psi \mid \phi) = \frac{\sum_{x \models \psi \wedge \phi} \prod_{x_i \in \mathbf{x}} w(x_i)}{\sum_{x \models \phi} \prod_{x_i \in \mathbf{x}} w(x_i)}.$$

What is the Problem?

$$p(\psi) = \sum_{x \models \psi} \prod_{x_i \in x} w(x_i)$$

How many worlds are we summing over?

For n Boolean variables:

$$|\Omega| = 2^n$$

For $n = 100$:

$$2^{100} \approx 10^{30}$$

possible worlds.

Marginalization

Suppose we have three Boolean variables

$$a, b, c.$$

We are interested in

$$p(a).$$

The values of b and c are irrelevant to the query.

Therefore we sum them out:

$$p(a) = \sum_{b,c} p(a, b, c).$$

This operation is called *marginalization*.

Marginalization as Forgetting

$$p(a) = \sum_{b,c} p(a, b, c)$$

Expanding the sum:

$$p(a) = p(a, \neg b, \neg c) + p(a, \neg b, c) + p(a, b, \neg c) + p(a, b, c).$$

Marginalization means forgetting variables we do not care about.

Marginalization over Possible Worlds

Recall:

$$p(\phi) = \sum_{x \models \phi} w(x).$$

To compute

$$p(a),$$

we must sum the weights of all worlds in which a is true.

$$p(a) = \sum_{x \models a} w(x).$$

Equivalently,

$$p(a) = \sum_{x \models a} \prod_{x_i \in x} w(x_i).$$

Why is Marginalization Hard?

Recall:

$$P(a) = \sum_{x \models a} \prod_{x_i \in x} w(x_i).$$

How many worlds must we sum over?

For n Boolean variables: $|\Omega| = 2^n$.
Computing a marginal may require summing over 2^{n-1} worlds.

Can we avoid enumerating all worlds?

A Simple SAT Problem

Consider the following formula in CNF (conjunctive normal form)

$$\phi = (u \vee v) \wedge (y \vee z).$$

Question:

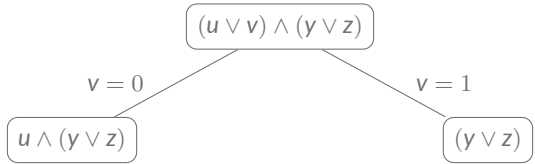
Does there exist a satisfying assignment?

SAT solvers answer this question by recursively splitting on variables.

DPLL Search

$$\phi = (u \vee v) \wedge (y \vee z)$$

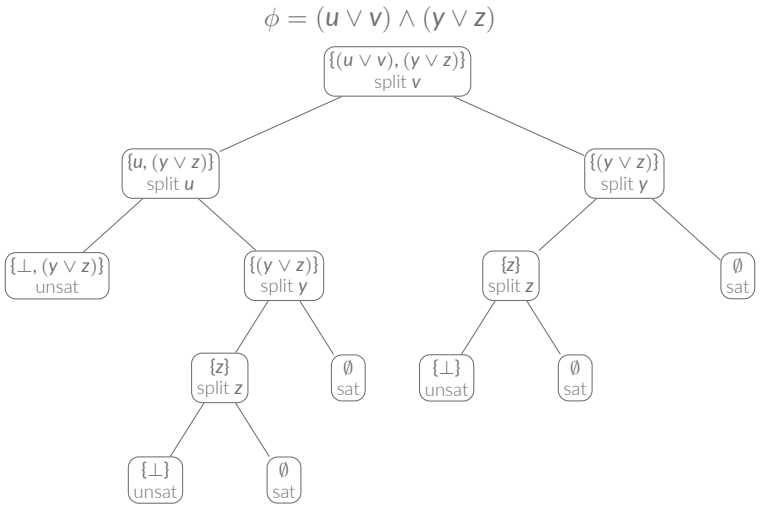
Choose variable v.



Substituting v yields:

$$\phi_{v=0} = u \wedge (y \vee z) \qquad \phi_{v=1} = y \vee z$$

DPLL Search Tree



DPLL as a Recursive Algorithm

Choose a variable v. Either

$$v = \top$$

or

$$v = \bot.$$

Therefore

$$SAT(\phi) = SAT(\phi_{v=1}) \ \vee \ SAT(\phi_{v=0}).$$

A SAT solver recursively explores the search tree until it reaches:

$$\top \qquad \text{or} \qquad \bot.$$

The DPLL Algorithm for SAT

DPLL = Davis–Putnam–Logemann–Loveland

Input:
a set of Boolean variables: $\mathit{Vars} = \{v_1, \ldots, v_n\}$
a CNF formula represented as a set of clauses: $S = \{C_1, \ldots, C_m\}$

Algorithm 1: DPLL

if $S = \emptyset$ **then**
  return $\top$
if S *contains an empty clause* **then**
  return $\bot$
select $v \in \mathit{Vars}$
$S_t \leftarrow S[v = \top]$
$S_f \leftarrow S[v = \bot]$
return DPLL($\mathit{Vars} \setminus \{v\}, S_t$) $\vee$ DPLL($\mathit{Vars} \setminus \{v\}, S_f$)

From SAT to #SAT

For SAT we ask:

$\exists$ satisfying assignment?

For #SAT we ask:

How many satisfying assignments exist?

The recursion changes from $\vee$ to $+$

$$\#SAT(\phi) = \#SAT(\phi_{v=1}) + \#SAT(\phi_{v=0}).$$

DPLL Variant for #SAT

Input:

$$Vars = \{v_1, \ldots, v_n\}, \qquad S = \{C_1, \ldots, C_m\}$$

Algorithm 2: #SAT

if $S = \emptyset$ **then**
  return $2^{|Vars|}$

if S *contains an empty clause* **then**
  return 0

select $v \in Vars$
$S_t \leftarrow S[v = \top]$
$S_f \leftarrow S[v = \bot]$
return #SAT$(Vars \setminus \{v\}, S_t) + $ #SAT$(Vars \setminus \{v\}, S_f)$

SAT uses $\vee$	*#SAT* uses $+$

DPLL Variant for Weighted Model Counting

Input:

$$Vars = \{v_1, \ldots, v_n\}, \qquad S = \{C_1, \ldots, C_m\}$$

Algorithm 3: Weighted Model Counting

if $S = \emptyset$ **then**
  return $\prod_{v \in Vars} (w(v) + w(\neg v))$
if S *contains an empty clause* **then**
  return 0
select $v \in Vars$
$S_t \leftarrow S[v = \top]$
$S_f \leftarrow S[v = \bot]$
return $w(v) \cdot \mathrm{WMC}(Vars \setminus \{v\}, S_t) + w(\neg v) \cdot \mathrm{WMC}(Vars \setminus \{v\}, S_f)$

The Same Search, Different Computations

For SAT:

$$SAT(\phi) = SAT(\phi_{v=1}) \ \vee \ SAT(\phi_{v=0})$$

For #SAT:

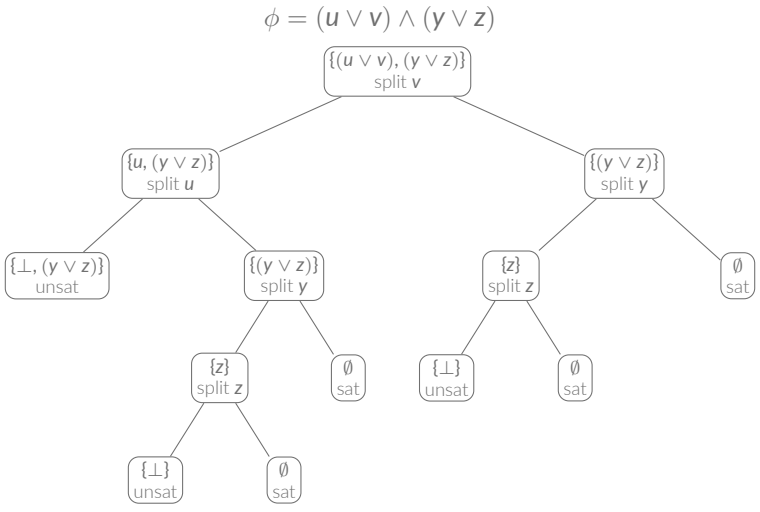
$$\#SAT(\phi) = \#SAT(\phi_{v=1}) + \#SAT(\phi_{v=0})$$

For Weighted Model Counting:

$$WMC(\phi) = w(v)\, WMC(\phi_{v=1}) + w(\neg v)\, WMC(\phi_{v=0})$$

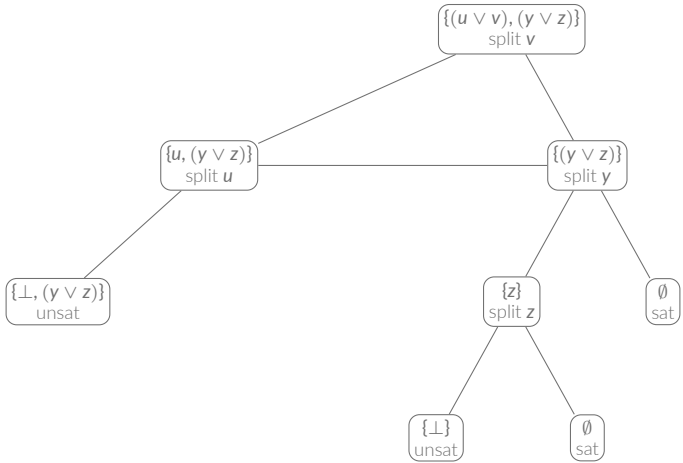
Same search tree. Different computations.

DPLL Search Tree



Caching Repeated Subproblems

$$\phi = (u \vee v) \wedge (y \vee z)$$



Component-Caching #SAT

Algorithm 4: Component-Caching #SAT

if $S = \emptyset$ then
    return $2^{|Vars|}$
if S *contains an empty clause* then
    return 0
if S *is in cache* then
    return cache[S]
select $v \in Vars$
$S_t \leftarrow S[v = \top]$
$S_f \leftarrow S[v = \bot]$
$r \leftarrow \#\text{SAT}(Vars \setminus \{v\}, S_t) + \#\text{SAT}(Vars \setminus \{v\}, S_f)$
Cache[S] $\leftarrow r$
return r

Independent Subproblems

Consider

$$\phi = (u \vee v) \wedge (y \vee z).$$

Notice that $\{u, v\}$ and $\{y, z\}$ do not share any variables.

Therefore we can solve the two parts independently.

$$\phi_1 = (u \vee v) \qquad \phi_2 = (y \vee z)$$

Counting Independent Components

Suppose

$$\phi = \phi_1 \wedge \phi_2$$

and the variables of ϕ_1 and ϕ_2 are disjoint.

Then every satisfying assignment of ϕ_1 can be combined with every satisfying assignment of ϕ_2.

Therefore

$$\#SAT(\phi) = \#SAT(\phi_1) \cdot \#SAT(\phi_2).$$

Example

$$\phi = (u \vee v) \wedge (y \vee z).$$

First component:

$$\#SAT(u \vee v) = 3$$

Second component:

$$\#SAT(y \vee z) = 3$$

Therefore

$$\#SAT(\phi) = 3 \times 3 = 9.$$

Two Fundamental Operations

Branching:

$$\#SAT(\phi) = \#SAT(\phi_{v=0}) + \#SAT(\phi_{v=1})$$

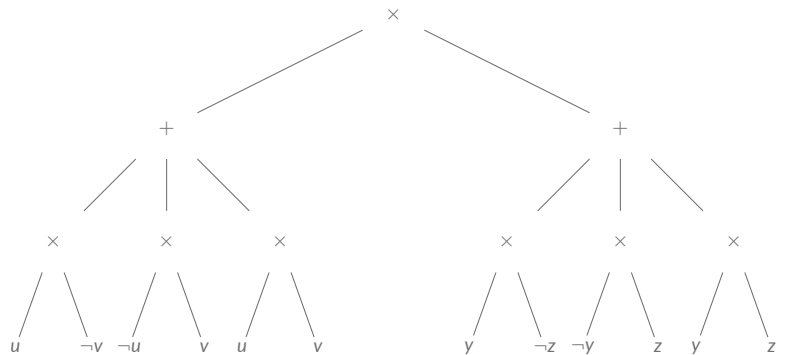
Independent subproblems:

$$\#SAT(\phi_1 \wedge \phi_2) = \#SAT(\phi_1) \cdot \#SAT(\phi_2)$$

when the variables are disjoint.

Counting using a sum-product computation graph

$$\phi = (u \vee v) \wedge (y \vee z)$$



What Does the DAG Compute?

The search DAG contains two operations:

Branching

$$\#SAT(\phi) = \#SAT(\phi_{v=0}) + \#SAT(\phi_{v=1})$$

Decomposition

$$\#SAT(\phi_1 \wedge \phi_2) = \#SAT(\phi_1) \cdot \#SAT(\phi_2)$$

sum and product are sufficient for counting.

WMC as Cached Circuit Evaluation

Algorithm 5: Eval(n)

if n *is in cache* **then**
  return cache[n]
if n *is a literal* ℓ **then**
  $r \leftarrow w(\ell)$
else if n *is a* $\times$*-node* **then**
  $r \leftarrow \prod_{c \in children(n)} \text{Eval}(c)$
else if n *is a* $+$*-node* **then**
  $r \leftarrow \sum_{c \in children(n)} \text{Eval}(c)$
cache[n] $\leftarrow r$
return r

We obtain the weighted model count by:

$$WMC(\phi) = \text{Eval}(root(\phi_{compiled}))$$